Strecht Simulados

Resumo de bolso Terraform Associate

Domínios cobrados, as dicas de prova mais importantes e 10 questões comentadas para treinar.

342	6	70%
questões no curso	provas completas	nota de corte

Como é a prova Terraform Associate

Infraestrutura como Código com Terraform

Fundamentos do Terraform

Fluxo de Trabalho Principal

Configuração do Terraform

Módulos do Terraform

Gerenciamento de State

Manutenção da Infraestrutura

HCP Terraform

Material independente de preparação, sem afiliação ou endosso de HashiCorp. As questões são inéditas e não reproduzem a prova real.

Pronto para o exame Terraform Associate?

Treine com as 342 questões em 6 provas completas: tempo do exame, explicação de cada alternativa e link da documentação para revisar.

Ver as 6 provas completas e o cupom do mês →

Dicas de prova por domínio

Configuração do Terraform

- Blocos aninhados repetidos: dynamic.
- Referência cria dependência implícita.
- Data source: data.TIPO.NOME.ATRIBUTO.
- Data source dependente: leitura no apply.
- Transformar lista: expressão for.
- default = null: opcional.
- auto.tfvars: o último em ordem alfabética vence.
- Instância de for_each: recurso["chave"].

Módulos do Terraform

- Mesmo módulo duas vezes: dois blocos.
- Ordem entre módulos: depends_on.
- Módulos irmãos: o raiz conecta.
- Módulo reutilizável: entradas enxutas.
- Pai para filho: argumentos do bloco module.
- Várias instâncias de módulo: for_each.
- Saída de módulo: module.NOME.OUTPUT.
- Módulo condicional: count 0 ou 1.

HCP Terraform

- Sem chaves longas: credenciais dinâmicas.
- Políticas em vários workspaces: policy sets.
- Acesso a um grupo de workspaces: projeto.
- Run manual: iniciar novo run.
- Bloco cloud + init: migração do estado.
- Commits disparam runs: fluxo VCS.
- Runs no mesmo workspace: fila.
- Módulos internos: registry privado.

Gerenciamento de State

- Locking: uma operação de escrita por vez.

- Aceitar desvio no estado: apply -refresh-only.
- Manter mudança manual: levar ao código.
- Mudança fora do Terraform: drift.
- HCP como backend: versões e lock.
- Backend local: lock do sistema de arquivos.
- Backend remoto: colaboração segura.
- Migrar estado: init -migrate-state.

Fluxo de Trabalho Principal

- PR com plan: impacto visível.
- apply sem arquivo: gera plano na hora.
- fmt -check: falha sem reescrever.
- Novo backend sem migrar: -reconfigure.
- Código de saída por mudanças: -detailed-exitcode.
- Plano aprovado: apply do arquivo.
- Checar a configuração sem nuvem: terraform validate.
- Write em equipe: controle de versão.

Manutenção da Infraestrutura

- Logs em arquivo: TF_LOG_PATH.
- import id: aceita expressões.
- Logs em JSON: TF_LOG=JSON.
- Renomear no estado pela CLI: state mv.
- Importar em massa: import com for_each.
- Desligar logs: remover TF_LOG.
- Falha interna: crash.log.
- Outputs para scripts: output -json.

Fundamentos do Terraform

- Versão exata: sem operador ou com =.
- Provider não declarado: hashicorp/NOME.
- Origem e versão de provider: required_providers.
- Valores aleatórios: provider random.
- Provider interno: espelho ou registry privado.

- Workspaces da CLI: estados separados.
- Estado: atributos, outputs e metadados.
- Plugins baixados: .terraform.

Infraestrutura como Código com Terraform

- Compliance: políticas sobre o código.
- Multi-cloud: referências entre providers.
- Testar antes: CI com validate e plan.
- Self-service seguro: módulos aprovados.
- Terraform: multi-provedor com um fluxo.
- Official: mantido pela HashiCorp.
- Alternativa ao HCL: .tf.json.
- Serviços variados: um provider para cada API.

10 questões para treinar

Responda antes de olhar o gabarito, no fim do material.

Questão 1 · Módulos do Terraform

Como fixar a versão de um módulo do Terraform Registry no bloco module?

A. Com o argumento version
B. Com o bloco required_providers
C. Com o argumento release
D. Com um sufixo no nome local

Questão 2 · Configuração do Terraform

Uma lista contém outras listas de sub-redes por zona. Qual função transforma tudo em uma única lista simples?

A. concat
B. distinct
C. flatten
D. compact

Questão 3 · Configuração do Terraform

Um recurso usa count com uma lista de nomes. Qual problema surge ao remover um nome do meio da lista?

A. O Terraform bloqueia a remoção por padrão
B. O estado é apagado e precisa ser importado
C. As instâncias seguintes mudam de índice
D. O item removido continua existindo para sempre

Questão 4 · Configuração do Terraform

Nas versões atuais do Terraform, a condição de validation de uma variável pode se referir a outras variáveis?

A. Não, só a data sources
B. Sim, desde o Terraform 1.9
C. Não, só ao próprio valor
D. Sim, mas só a outputs

Questão 5 · Infraestrutura como Código com Terraform

Uma equipe cria ambientes de teste que só são usados durante o dia. Como a IaC ajuda a controlar custos?

A. Manter todos os ambientes ligados sem uso
B. Desligar a cobrança do provedor de nuvem
C. Reduzir o preço por hora dos recursos usados
D. Recriar e destruir ambientes com facilidade

Questão 6 · Módulos do Terraform

Qual estrutura de arquivos é convencional em um módulo reutilizável?

A. module.json, vars.json e out.json
B. main.tf, variables.tf e outputs.tf
C. init.tf, plan.tf, apply.tf e run.tf
D. state.tf, lock.tf e backend.tf

Questão 7 · Módulos do Terraform

Qual faixa de versões a restrição ~> 1.2.0 permite para um módulo?

A. De 1.2.0 até antes de 1.3.0
B. De 1.2.0 em diante, sem teto
C. De 1.2.0 até antes de 2.0.0
D. Exatamente a versão 1.2.0

Questão 8 · Gerenciamento de State

Um recurso foi retirado do código e do estado, mas continua existindo na nuvem. Como o Terraform o trata depois disso?

A. Destrói o recurso no próximo plan
B. Não o vê mais nem o gerencia
C. Recria o recurso no próximo apply
D. Importa o recurso automaticamente

Questão 9 · Configuração do Terraform

Um data source consulta um recurso criado na mesma configuração, referenciando seu id. Como fica a ordem?

A. O data source é lido antes de tudo
B. O Terraform acusa ciclo de dependência
C. O recurso é criado antes da leitura
D. Os dois ocorrem sem ordem definida

Questão 10 · Configuração do Terraform

Qual função lê um arquivo de modelo e substitui variáveis nele, útil para gerar scripts de inicialização?

A. filebase64
B. formatlist
C. templatefile
D. jsondecode

Gabarito comentado

1. A) Com o argumento version
Versão de módulo do registry: version.

2. C) flatten
Listas aninhadas: flatten.

3. C) As instâncias seguintes mudam de índice
count com lista mutável: prefira for_each.

4. B) Sim, desde o Terraform 1.9
validation cruzada: Terraform 1.9+.

5. D) Recriar e destruir ambientes com facilidade
Custo: ambientes efêmeros.

6. B) main.tf, variables.tf e outputs.tf
Módulo padrão: main, variables, outputs.

7. A) De 1.2.0 até antes de 1.3.0
~> 1.2.0: só patches.

8. B) Não o vê mais nem o gerencia
Fora do código e do estado: órfão.

9. C) O recurso é criado antes da leitura
Data source que referencia resource: depois dele.

10. C) templatefile
Modelo com variáveis: templatefile.

Pronto para o exame Terraform Associate?

Treine com as 342 questões em 6 provas completas: tempo do exame, explicação de cada alternativa e link da documentação para revisar.

Ver as 6 provas completas e o cupom do mês →